

LLM 预训练之 Sequence 构造

Sequence Construction • 深度研究手册

从 Tokenized Documents 到训练样本：Document Boundary、Concat-and-Chunk、Best-fit Packing、Intra-document Masking、Variable Length 与 Length Curriculum

核心视角：Sequence Builder 在“计算利用率、文档完整性、上下文语义、优化稳定性、长上下文能力”之间做联合设计

Research cut-off: 2026-08-13 | 01-B-Agent

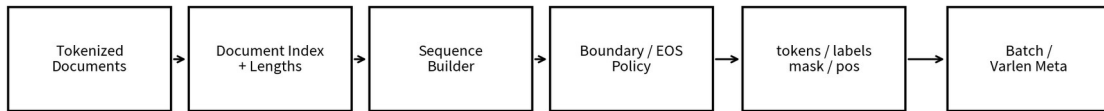
执行摘要：Sequence Construction 是预训练样本的“编译阶段”

Tokenizer 只告诉我们每篇文档由哪些 token 构成；Transformer 真正收到的却是长度受限的训练 sequence。中间这一层负责决定：文档以什么顺序进入样本、是否被切断、多个文档是否拼在一起、边界处是否插入 EOS/EOD、跨文档 attention 是否允许、position 是否重置、哪些 token 计入 loss，以及一个 batch 的有效 token 利用率。

核心判断	Know-why	工程含义
固定长度并非唯一正确答案	Attention 成本依赖真实长度，而数据长度高度不均	Variable-length / bucket / curriculum 可直接改变训练效率
Packing 效率与语义完整性存在张力	Concat-and-chunk 可 100% 填满，但会切断文档并产生跨文档上下文	需要区分 padding waste、fragmentation 与 contamination
同一 pack 可有不同 attention 语义	普通 causal mask 允许后文档读前文档；block/intra-doc mask 不允许	“拼接”与“上下文关系”必须分开设计
Sequence length 是 curriculum	长序列早期可能增加梯度方差与训练不稳定	可 warmup 长度或分阶段长上下文 CPT
文档顺序也是能力变量	随机相邻文档通常互不相关；相关文档可形成可利用上下文	In-context pretraining / retrieval-based construction

最重要的 Know-why Sequence Builder 实际定义了模型在预训练时看到的“局部世界”。即使 token 集合完全一样，仅改变这些 token 如何分组、排序和互相可见，也能改变语言建模、上下文利用、知识记忆和长上下文能力。

Sequence 构造是 Tokenizer 与 Transformer 之间的“训练样本编译器”



Sequence Construction 决定：哪些 token 同处一个训练上下文、文档是否被切断、跨文档是否可见、哪些 token 计入 loss、每步真实计算多少有效上下文

图 1 Sequence Construction 的系统位置：它把 tokenized documents 编译为 tokens / labels / masks / positions / varlen metadata。

0. 问题定义：从文档集合到可训练 sequence

设文档 $d_j = (t_1, \dots, t_{n_j})$ ，模型最大训练长度为 L 。Sequence Construction 要构造样本 s_k ，满足长度约束，同时定义边界与监督语义。至少需要同时优化四类目标：

目标	一个可测指标	如果忽略
计算效率	packing efficiency = useful tokens / allocated slots	大量 PAD、吞吐低、显存浪费
内容完整性	document fragmentation / truncation rate	长距离逻辑、代码结构、事实一致性被切断
上下文语义	cross-document attention / relatedness	模型被迫利用随机无关前文
训练分布	sequence-length histogram / tokens by length bucket	模型只在固定单一长度上学习，长上下文泛化弱
监督正确性	loss-mask coverage / boundary loss policy	PAD、prompt、EOD 等不该训练的位置进入 loss

一个常见误解是把“context length = 8192”理解成“模型每个样本都在学习 8192-token 的连贯上下文”。如果 8192 token 由 20 个随机网页片段拼接而成，并且跨文档被 mask，那么真实可利用的连续上下文长度可能远小于 8192。

1. 基线：Concat-and-Chunk 为什么如此普遍

现代 decoder-only 预训练的经典基线是：打乱文档 -> 每篇末尾插 EOD/EOS -> 连续拼成长 token stream -> 每 L（常再取 L+1）个 token 切成一个训练样本。Megatron Core 的 GPTDataset 就显式维护 document index、sample index 和 shuffle index；样本可跨多个文档，并通过额外 1 个 token 形成同长的输入 tokens 与 shifted labels。[9]

步骤	典型实现	目的
Document shuffle	构造 document_index	避免固定语料顺序形成偏差
Concatenate	允许样本跨文档取 token	消灭 padding、接近 100% token 利用
Chunk L+1	input = x[:-1], label = x[1:]	得到 L 个 next-token targets
EOD/EOS	文档末尾特殊 token	显式标记边界；可选择是否在该位置计 loss
Sample shuffle	shuffle_index	训练时随机化固定 chunk 的访问顺序

为什么它成功 它非常容易流式化、索引化、缓存和随机访问，几乎不浪费 token slots，而且与标准 causal attention 完全兼容。这是一个“系统效率优先”的优雅 baseline。

四种常见 Sequence Construction 策略：效率、完整性与 attention 语义不同

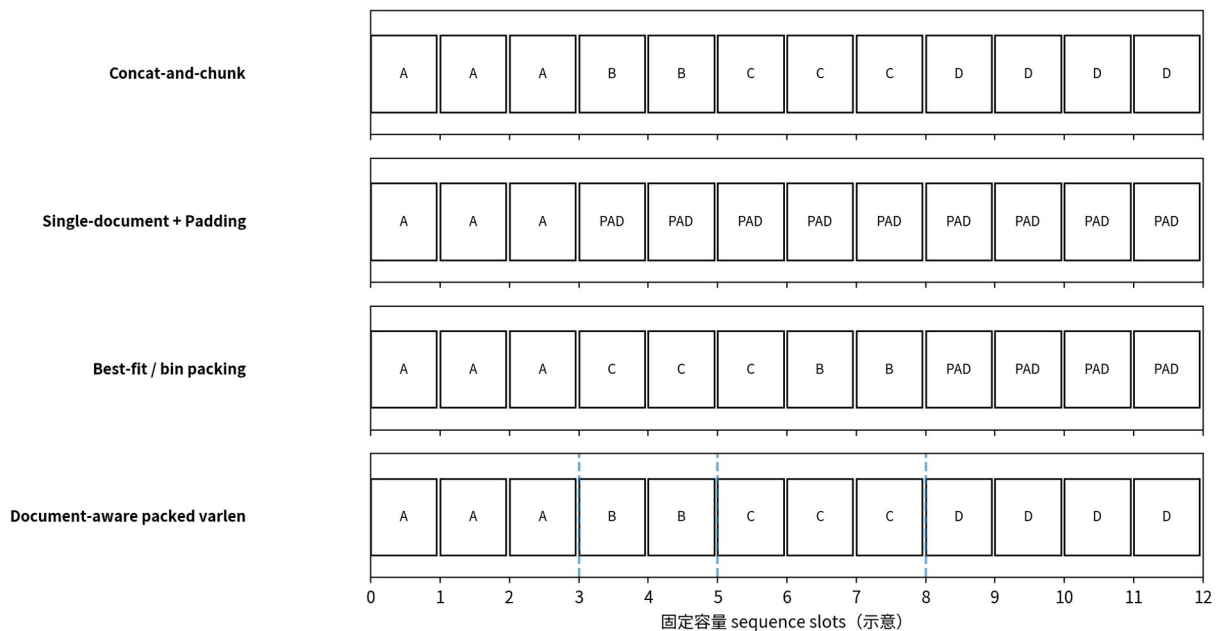


图 2 常见序列构造方法的概念比较。实际实现还会加入 EOS、mask、position 与 batch 约束。

2. Concat-and-Chunk 的三个隐性代价

代价	机制	为什么会伤能力	代表证据
Fragmentation	固定边界可能把一个文档切成前后两段	模型学习到的长程语义被截断	Best-fit Packing [2]

代价	机制	为什么会伤能力	代表证据
Cross-document distraction	Doc B token 的 causal prefix 含随机 Doc A	前文与目标无关，形成噪声条件	Sequence Composition [3]
Effective length shrink	多个短文档被 block mask 后，计算仍按 pack/固定 L 组织	有效 attention 长度远小于配置 L	Dataset Decomposition [1]

“Fewer Truncations Improve Language Modeling” 将固定 concat-and-chunk 的问题归因于不必要的文档截断，并提出 Best-fit Packing；论文摘要报告阅读理解相对 +4.7%、context following +16.8%、program synthesis +9.2%，以及 closed-domain hallucination 最多降低 58.3%。这些是该论文在其实验设置中的结果，不应直接外推为所有模型的固定收益。[2]

“Analysing The Impact of Sequence Composition” 则从另一方向指出：普通 causal masking 会让后一个文档条件化在前一个无关文档上；其 intra-document causal masking 显著改善语言建模和下游表现。[3]

拼在同一 sequence ≠ 必须互相 Attention：边界语义由 mask / cu_seqlens 决定

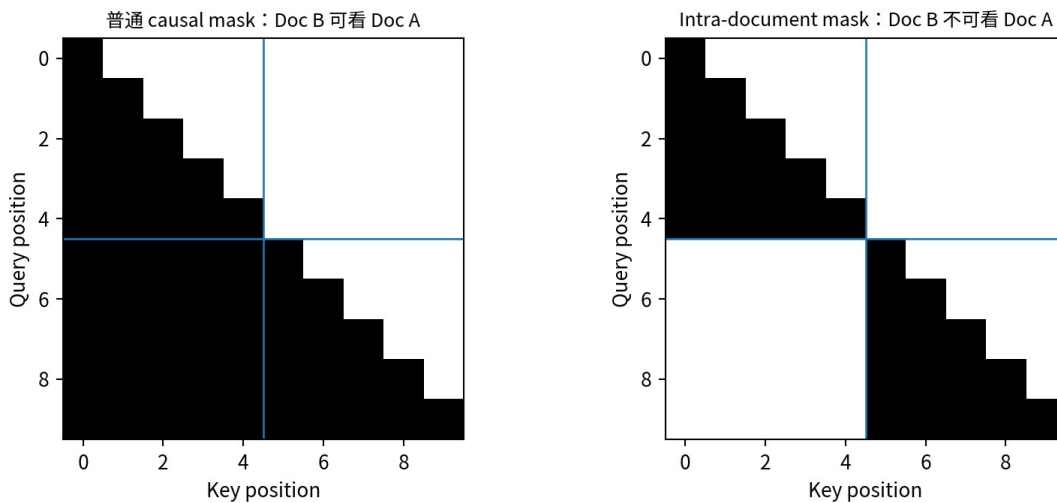


图3 “同一 pack 内是否跨文档可见”是独立于 packing 本身的设计维度。

3. 边界语义：EOS、Attention Mask、Position Reset、Loss Mask

Megatron Core 当前 GPTDataset 暴露 reset_position_ids、reset_attention_mask、eod_mask_loss 与 inter_document_masking 等开关：这恰好说明一个训练 sample 中存在多个文档时，至少有四种语义需要独立决定。[9]

边界机制	可选策略	影响
EOD/EOS token	每文档插入 / 不插入	模型是否显式看到“文档结束”事件
Attention boundary	允许跨文档 / block-diagonal intra-doc	决定 Doc B 能否读 Doc A
Position IDs	连续累计 / 每文档 reset	模型把拼接看成一个长序列还是多段独立序列
Loss on EOD	训练 / mask out	是否要求模型学习预测文档终止
Loss on PAD	必须 mask	否则模型学会预测 padding token

Know-how 不要用一个布尔变量 “packed=true” 概括所有行为。生产配置应显式版本化：packing policy、boundary token policy、attention boundary、position reset、loss mask 和 kernel path。否则换框架后很容易得到“同名 packing、不同训练目标”。

当 inter-document masking 启用时，Megatron 会返回文档边界的 `cu_seqlens`，并按文档重置 position IDs；FlashAttention 的 `varlen` 接口也以 `cu_seqlens` 和 `max_seqlen` 作为核心输入，使 packed tokens 能按独立子序列计算 attention，而无需显式构造大 block mask。[9][10]

4. Padding vs Packing：先区分训练数据形态

策略	Padding waste	文档完整性	跨文档污染	系统复杂度	典型适用
单文档 + pad to L	高，尤其长度分布长尾时	高	无	低	评测/简单训练
Dynamic pad to batch max	中	高	无	中	变长 batch
Concat-and-chunk	近零	低-中	默认有	低	大规模预训练 baseline
Bin / Best-fit packing	低	高	取决于 mask	中-高	保完整性并降低 PAD
Varlen packed + <code>cu_seqlens</code>	低	高	可严格隔离	高，需要 kernel 支持	现代 GPU 高效 path

早期 sequence packing 工作把问题形式化为 bin packing，并指出 variable-length NLP 数据中 padding 可占相当高比例；通过不发生 cross-contamination 的 packing，在 BERT phase-2 pretraining 上展示约 2x speedup。该证据来自 encoder/BERT 场景，但 bin-packing 与隔离 attention 的工程思想延续至现代 LLM。[8]

NeMo 的 packed-sequence 文档进一步说明：若只是把多条序列拼起来再构造显式 block triangular mask，计算可能仍按总 pack 长度平方增长；使用 FlashAttention/TransformerEngine 的 variable-length kernel，可让复杂度更接近各子序列 attention 成本之和，并通过 `cu_seq_lens` 传递边界。[11]

5. Best-fit Packing：目标不是“塞满”，而是“少切文档”

传统 concat-and-chunk 的 packing efficiency 很高，但它本质上把 token stream 当作连续材料切割。Best-fit 思路则把文档或可接受的完整片段视为 item，把每个 sequence 视为容量为 L 的 bin，用长度感知的组合优化减少未填充空间，同时尽量不切断 item。

设计点	简单做法	更稳健做法
Item 定义	整篇文档	对超长文档按结构边界先切成语义完整 segment
排序	随机	length-aware / best-fit decreasing + 随机扰动
剩余空间	PAD	选择最接近 remaining capacity 的 item
文档相关性	不考虑	可结合 domain / retrieval / source constraints
跨 item attention	默认可见	按任务目标选择 intra-doc mask 或相关文档可见

关键 trade-off 完全不切文档可能造成更高 padding 或更复杂 packing；完全追求 100% fill 又可能牺牲文档完整性。优化目标应是“有效训练效用 / FLOP”，而不是单独最大化 packing ratio。

6. Intra-document Causal Masking：把“物理拼接”与“逻辑上下文”解耦

设一个 pack 由长度 $l_1, \dots, l_m$ 的文档组成。物理上可把它们连续存储为一个 tensor；逻辑上 attention 可以是多个独立 causal block。若 kernel 支持 varlen，这种表示能同时获得低 padding 与无跨文档污染。

传统显式大 mask 的计算形状可能仍受 $(\sum l_i)^2$ 影响；varlen kernel 的目标则是只计算各 block 的有效 attention，使成本更接近 $\sum l_i^2$ 。

方案	Token storage	Attention graph	Position	优点
Naive concatenation	连续	单一全局 causal triangle	连续	最简单
Block mask	连续	多个 causal blocks	可连续/重置	语义正确但 kernel 需高效支持
Varlen cu_seqlens	连续 flat THD	kernel 直接按边界分段	通常按子序列处理	避免无效跨 block attention

Sequence Composition 论文还进一步提出 BM25Chunk：不是简单隔离随机文档，而是检索相关文档组成上下文；摘要报告 in-context learning +11.6%、knowledge memorisation +9.8%、context utilisation +7.2%。这说明“跨文档 attention”不一定有害，真正的问题是跨的是不是有意义的文档。[3]

7. In-context Pretraining：Sequence Ordering 本身可以成为训练信号

Meta 等提出 In-Context Pretraining，将随机短文档拼接改为“相关文档放在同一上下文”。他们通过近邻检索与图遍历生成相关文档序列，不重复数据，只改变排序；摘要报告 in-context learning +8%、reading comprehension +15%、faithfulness +16%、long-context reasoning +5%、retrieval augmentation +9%。[4]

随机 concat	Related-document construction
前文档通常对后文档无预测价值	前文档可能提供主题、实体或事实背景
模型学到“忽略无关前文”也是合理策略	模型被鼓励跨文档利用上下文
实现最简单、无检索成本	需要全语料相似度/图构建，数据管线更重
适合把文档看作独立样本	适合培养 ICL、检索增强和多文档推理

Know-why Sequence Construction 不只是减少 PAD 的“压缩器”，也可以是一种弱监督：通过把相关内容放在一起，向模型暗示“前文值得使用”。

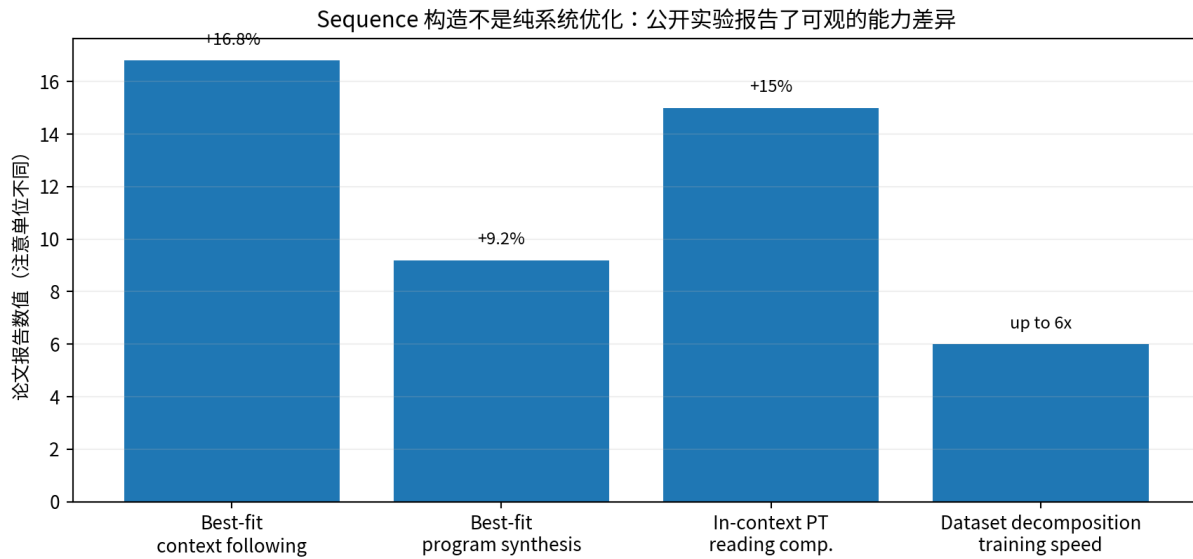


图 4 若干论文在各自设置下报告的代表性结果。柱高单位并不统一，图的目的仅是展示“sequence design 会影响能力”，不能横向比较柱高。

8. Variable Sequence Length：固定 L 会为短文档支付不必要的 attention 成本

Apple 的 Dataset Decomposition 指出，经典固定长度数据通常由随机 concat-and-chunk 构造；如果实际文档很短、又使用跨文档 masking，则很多训练计算并未换来更长的有效上下文。他们把数据分解为多个同长度 bucket，并在训练时用可变 sequence length 与 batch size 共同采样。[1]

固定长度训练	Variable-length / bucket 训练
所有 step 接近相同 L	step 可来自不同长度 bucket
短文档仍占用固定 attention shape	成本更贴近真实 sequence length
batch size 通常固定或受 max L 限制	可按 token/FLOP budget 动态调 batch size
数据构造简单	scheduler / loader / kernel 更复杂

该论文摘要报告：在相近成本下训练 8K context 的 1B 模型，可达到固定 2K baseline 的成本量级，并在其 web-scale 实验中达到目标准确率最多约 6x 更快。[1]

工程原则 如果目标是“每 step 固定 token 数”，variable L 时 batch_size 应大致与 1/L 反向调整；如果目标更接近“固定 attention FLOPs”，则应考虑 L^2 项，batch 调度更激进。真正实现还要受 activation memory、kernel tile 和并行策略约束。

9. Sequence Length Warmup：长序列是优化变量，不只是数据变量

Sequence Length Warmup 的实验把训练不稳定与长 sequence 引起的极端 gradient variance 关联起来，并提出训练早期使用短 sequence、随后逐步增长。摘要报告在复现 GPT-2/GPT-3 小模型的设置中，可支持更大 batch / learning rate，并将达到同等或更好 zero-shot 结果所需 token 和 wall-clock 最多分别降低 2.2x 与 3.7x。[5]

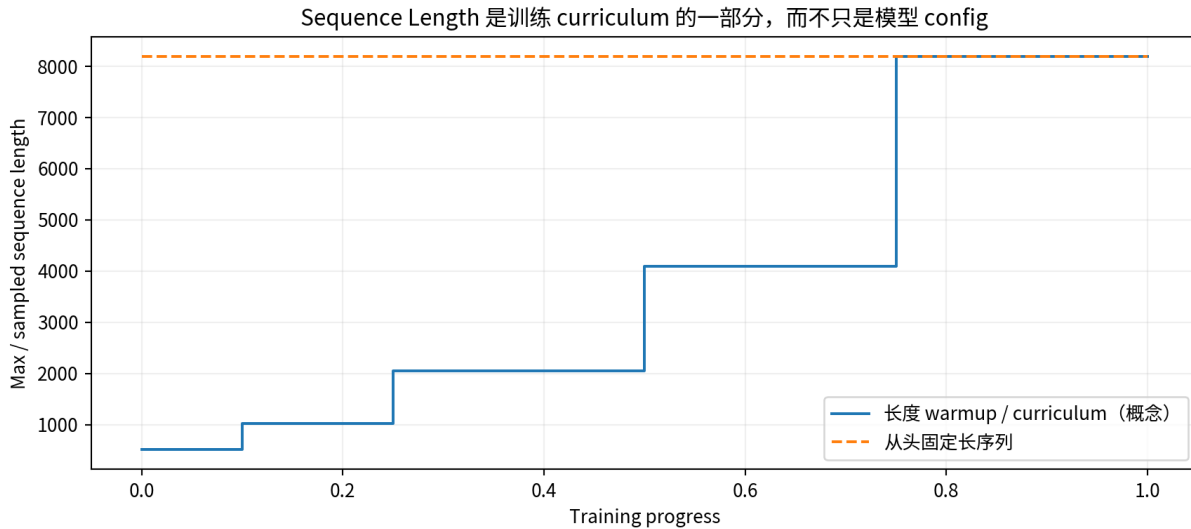


图 5 Sequence length curriculum 概念图；曲线仅示意，不对应某篇论文的具体 schedule。

阶段	常见长度策略	主要目的
Early	短 L / 多 batch	降低 gradient variance、提高吞吐、快速学局部统计
Middle	逐步提高 L	迁移到中距离依赖
Late	目标 L / 长文本比例提高	训练长上下文位置与长依赖
Long-context CPT	从已训练 base 继续更长 L	比从零开始一直用超长序列更省算力

10. 长上下文训练：Max Length ≠ Effective Context

Effective Long-Context Scaling 从 Llama 2 继续预训练到最高 32K，并研究长文本 upsampling 和 sequence-length curriculum；其结果强调“有大量长文本”并不是强长上下文能力的唯一关键，而且从 base model 做长上下文 continued pretraining 可比从头用长序列训练更经济。[6]

ProLong 的研究进一步指出：长上下文 continued pretraining 中，code repositories 和 books 是有价值的长数据来源，但必须与高质量短数据混合；训练长度超过最终评测长度也可能提高长上下文表现。[7]

必须分别报告	为什么
Model max context	架构/位置编码允许的上限
Train max sequence length	训练真正见过的最大长度
Sequence-length distribution	多少 token 真正在 2K/8K/32K/128K 等长度段训练
Document span distribution	sequence 内是否是一篇完整长文，还是许多短文拼接
Attention boundary policy	跨文档是否真正形成长依赖
Long-text source mix	代码、书籍、论文、网页的长结构不同

最常见的误读 “模型训练长度 128K” 并不自动意味着模型学会了 128K 依赖。若绝大部分 sequence 是随机短文档拼接、且跨文档隔离，则它见到的是大容器中的许多短任务。

11. Shuffling：在什么粒度随机化？

Sequence Construction 还包含一个经常被隐藏的变量：先拼再 shuffle，还是先切成 atom 再 shuffle；atom 是 document、segment 还是完整 sequence。不同顺序会同时影响局部连贯性和样本独立性。

2024 年一项 packing/shuffling 研究对 concatenation 与 padding 的 atom size 做实验，报告在其生成式 LM 设置下，atom size 与最大 sequence length 匹配表现较好；padding 取得更低 perplexity，但代价是更多训练 step 和更低 compute efficiency。该结论更适合作为“需要做 ablation”的证据，而不是通用定律。[12]

Shuffle unit	优点	风险
Document	保持文档内部顺序	长文档可能形成连续长段，sample 分布不均
Fixed sequence	读取简单、训练样本 iid-like	如果 sequence 本身是随机 concat，边界语义已固定
Segment / atom	可控制局部连贯性与粒度	atom size 成为新超参
Related-document group	培养跨文档利用	检索偏差、topic clustering 可能降低多样性

12. Code / Math / Structured Data：不要统一按自然语言切

数据类型	不合适的切法	更好的 Sequence Construction 思路
Code repo	随机 token chunk 切断函数/类/文件依赖	优先 file/module/function 边界；保 path/import/neighbor context；超长 repo 做结构化 segment
Math / proof	任意截断证明链	按题目-解答/定理-证明结构保持闭合
Books	每 L token 硬切	章节/段落边界 + best-fit；保留部分长连续 chapter
Scientific papers	跨 section 随机截断	title/abstract/section/formula/table metadata 结构化构造
Dialogue / instruction	跨样本混 attention	pack 可物理拼接，但必须明确 sample boundary 与 loss mask

这里的原则不是“永远不切”，而是把结构边界作为高价值 cut candidates。超长文档仍然需要 chunk，但 chunk 应尽量与真实语义单元一致，并保留 parent metadata 供采样与后续分析。

13. Production Blueprint：把 Sequence Builder 做成可审计组件

阶段	输入	输出 / 关键字段
1. Document Reader	token ids + doc_id + source metadata	doc_len, quality, domain, language
2. Segmentation	超长 document	segment_id, parent_doc_id, structural_boundary
3. Ordering	documents / segments	shuffled / related / curriculum order
4. Packing	items + target L	pack_id, item offsets, padding count
5. Boundary Compiler	pack	EOD positions, cu_seqlens, attention policy
6. Supervision Compiler	tokens	labels, loss_mask, ignored positions
7. Position Compiler	boundaries	position_ids / reset policy
8. Batch Scheduler	sequence lengths	microbatch, token budget, bucket id
9. Telemetry	all above	packing efficiency, fragmentation, length histogram, cross-doc stats

推荐的 Canonical Record 不要只落盘 input_ids。至少保留 doc/segment boundaries、source IDs、EOD positions、loss policy、sequence policy version 与原始 item lengths。这样才能复现实验、做 contamination/debug、重建不同 attention policy。

14. 该监控什么：Sequence Observability Scorecard

指标	定义/直觉	异常意味着什么
Packing efficiency	non-pad tokens / allocated tokens	低 -> GPU 在算 PAD
Fragmentation rate	被跨 sequence 切开的文档比例	高 -> 长文档完整性差
Fragments per doc	每文档平均训练片段数	高 -> 同文档上下文高度碎片化
Cross-doc token ratio	sequence 中非首文档 token 占比	结合 mask 判断随机前文暴露程度
Docs per sequence	每个 sample 文档数分布	过高 -> nominal L 与 effective context 脱节
Effective attention length	按 mask 后实际子序列长度统计	长上下文训练是否真实发生
Length histogram	tokens/steps by sequence-length bucket	curriculum 是否按计划执行
Boundary loss rate	EOD/PAD/ignored token 的 loss 比例	监督 policy 是否实现正确
Repeat adjacency	同 source/domain 连续相邻概率	可能存在排序偏差

15. Ablation：怎样证明某种 Sequence 构造真的更好

Sequence 策略很容易与“实际 token 数、有效 FLOPs、长文本比例、训练 wall-clock”纠缠。一个可信 ablation 至少需要三套匹配条件。

对照口径	固定什么	回答的问题
Fixed raw tokens	训练同样数量的 non-pad tokens	数据效用是否更高？
Fixed model FLOPs	约束 attention/MLP 总计算	同等算力谁更强？
Fixed wall-clock	同集群同时间	生产吞吐 + 能力综合谁更优？
Fixed document set	完全相同 doc，只改构造	sequence policy 本身的因果效应

- 至少跑：concat-and-chunk baseline、document-preserving packing、intra-document masking、variable-length 或 length curriculum。
- 评测不能只看 validation loss：加入 long-context、context following、reading comprehension、code completion、memorization/contamination 等。
- 同时记录 tokens/sec、MFU、padding ratio、attention FLOPs 与数据 loader CPU 开销；否则“更快”可能只是改变了真实训练量。

判断标准 最优策略不是 packing efficiency 最高，也不是 perplexity 最低，而是在目标模型规模、训练 horizon、目标能力和系统约束下，让“单位算力带来的最终能力”最大化。

16. 设计决策树与 Know-how Checklist

问题	如果 Yes	如果 No
大量文档远短于 L？	优先 packing / varlen，避免 padding	可简化 batching
必须保持文档完整？	best-fit / structure-aware segmentation	concat-and-chunk 可作为 baseline
多个文档互不相关？	intra-doc mask / cu_seqlens	相关文档可考虑跨文档 attention
目标包含 ICL/多文档推理？	related-document ordering 做专项 ablation	随机 shuffle 足够

问题	如果 Yes	如果 No
目标 context 很长?	length histogram + long CPT + long source mix	无需为超长序列长期付出 $O(L^2)$
训练早期不稳定?	sequence-length warmup 是候选杠杆	保持固定长度并检查其他优化项
kernel 支持 varlen?	优先 boundary metadata 直达 kernel	显式 block mask 需检查实际 FLOPs

最终 Know-why Sequence Construction 是一种“上下文分布工程”：Data Mixture 决定模型看哪些文档，Tokenizer 决定文档变成哪些 token，而 Sequence Builder 决定这些 token 在每一次预测时彼此处于什么上下文关系。三者共同定义真正的 pretraining distribution。

参考资料与证据等级

- [1] Pouransari et al., Dataset Decomposition: Faster LLM Training with Variable Sequence Length Curriculum, 2024 | A - 原始论文 [source](#)
- [2] Ding et al., Fewer Truncations Improve Language Modeling, 2024 | A - 原始论文 [source](#)
- [3] Zhao et al., Analysing The Impact of Sequence Composition on Language Model Pre-Training, 2024 | A - 原始论文 [source](#)
- [4] Shi et al., In-context Pretraining: Language Modeling Beyond Document Boundaries, 2023 | A - 原始论文 [source](#)
- [5] Li et al., The Stability-Efficiency Dilemma: Sequence Length Warmup for Training GPT Models, 2021 | A - 原始论文 [source](#)
- [6] Xiong et al., Effective Long-Context Scaling of Foundation Models, 2023 | A - 原始论文 [source](#)
- [7] Gao et al., How to Train Long-Context Language Models (Effectively), 2024 | A - 原始论文 [source](#)
- [8] Krell et al., Efficient Sequence Packing without Cross-contamination, 2021 | A - 原始论文 [source](#)
- [9] NVIDIA Megatron Core GPTDataset - document/sample/shuffle indices, EOD masks, inter-document masking | A - 官方实现文档 [source](#)
- [10] Dao-AI Lab FlashAttention - variable-length attention interface (cu_seqlens) | A - 官方代码 [source](#)
- [11] NVIDIA NeMo Sequence Packing - variable-length kernel / cu_seq_lens explanation | A - 官方框架文档；主要是 SFT 证据 [source](#)
- [12] Chen et al., Refining Packing and Shuffling Strategies for Enhanced Performance in Generative Language Models, 2024 | B - 原始论文，规模/设置有限 [source](#)
- [13] Yao et al., Hierarchical Balance Packing: Efficient SFT for Long-Context LLM, 2025 | B - 原始论文；SFT 系统侧前沿 [source](#)

证据说明：本文优先使用原始论文、官方实现和官方框架文档。SFT/PEFT 的 packing 资料仅用于解释 kernel/packing 机制，不直接当作“预训练能力收益”的证据。图 2、图 3、图 5 为机制示意；图 4 中数值来自对应论文摘要，但单位不同，因此不可跨柱比较。